Avance de Proyecto: Diseño y Análisis

Fernando Israel Rios Garcia-AL02877016

Universidad Tecmilenio-Campus Corregidora

LSTI2307: Programación Orientada a Objetos

Ricardo Monrroy

30 de enero de 2026

Análisis de Requerimientos

Objetivo

El sistema busca eficientizar y automatizar la gestión administrativa de una biblioteca mediante una solución de software desarrollada en Java. El objetivo principal es facilitar el registro y consulta de libros, usuarios y préstamos, asegurando la integridad de los datos a través de validaciones robustas y reglas de negocio, como el control de stock y límites de préstamo por usuario. Asimismo, el sistema está diseñado para proveer reportes claros sobre el estado operativo de la biblioteca, priorizando la facilidad de uso, eficiencia y una arquitectura escalable que permita futuras expansiones, todo ello en cumplimiento con los lineamientos académicos de la materia LSTI2307.

Requisitos Funcionales

El sistema cumple con las siguientes funciones esenciales:

- **Gestión de Entidades**: Registro y consulta de libros, usuarios y préstamos.
- **Validación de Datos**: Verificación de coherencia y corrección en la información ingresada.
- **Reglas de Negocio**: Control de stock (no prestar libros sin disponibilidad) y límite de préstamos (máximo dos ejemplares por usuario).
- **Reportes**: Generación de informes en pantalla sobre el estado general de la biblioteca, incluyendo conteos de libros, usuarios y préstamos activos.

Diseño de Clases y Objetos

A continuación se detallan las clases principales identificadas en el modelo orientado a objetos, las cuales representan las entidades fundamentales del dominio de la biblioteca.

Clase Principal

Library Actúa como el controlador principal del sistema.

- **Atributos**: Listas de usuarios (`users`), materiales (`materials`), mobiliario (`furniture`) y préstamos (`loans`).
- **Métodos**: Permite registrar usuarios, materiales y préstamos (`registerUser`, `registerLoan`), así como generar reportes generales (`generateReport`).

Entidad Base

LibraryEntity (Abstracta) Clase base para todos los objetos gestionables en la biblioteca.

- **Atributos**: Identificador (`id`), nombre (`name`) y fecha de creación (`createdAt`).
- **Métodos**: Validación básica de la entidad (`validate`).

Jerarquía de Usuarios

Heredan de `LibraryEntity`.

User Representa a un usuario genérico.

- **Atributos**: Apellidos, edad, email, balance y préstamos activos.
- **Métodos**: Verificar elegibilidad (`canBorrow`), agregar o remover préstamos.

Visitor Subclase de Usuario. Tiene un límite de préstamos predefinido (e.g., 2 ítems).

Librarian Subclase de Usuario. Posee un rol administrativo y métodos para registrar libros y usuarios en el sistema.

Jerarquía de Materiales

Heredan de `LibraryEntity`.

Material Clase base para los recursos prestables.

- **Atributos**: Autor, fecha de publicación, stock y condición.
- **Métodos**: Verificación de disponibilidad (`isAvailable`) y gestión de stock.

Book Atributos específicos: ISBN, editorial, categoría en estante.

Magazine Atributos específicos: Volumen, número.

DVD Atributos específicos: Duración, región.

Jerarquía de Mobiliario

Heredan de `LibraryEntity`.

Furniture Clase base para el inventario físico no prestable.

- **Atributos**: Fecha de ingreso y estado operativo (`isOperational`).

Shelf Estantería. Define capacidad y sección.

Desk Escritorio. Define capacidad de asientos y ubicación.

Chair Silla. Define material, si es ergonómica y si tiene ruedas.

Gestión de Préstamos

Loan Modela la transacción de préstamo.

- **Atributos**: Fecha de inicio, fecha de vencimiento y estado.
- **Métodos**: Cerrar préstamo (`closeLoan`) y verificar si está vencido (`isOverdue`).

Relación entre Clases

La arquitectura del sistema se basa en las siguientes relaciones estructurales:

Herencia (Generalización)

Se ha implementado una jerarquía robusta para reutilizar código y agrupar comportamientos comunes:

- **LibraryEntity** es la clase padre de `User`, `Material` y `Furniture`. Esto permite que todas las entidades compartan propiedades básicas como ID y fecha de registro.

- **User** es padre de `Visitor` y `Librarian`.
- **Material** es padre de `Book`, `Magazine` y `DVD`.
- **Furniture** es padre de `Shelf`, `Desk` y `Chair`.

Composición (Todo-Parte)

La clase **Library** mantiene una relación de composición fuerte con las entidades del sistema. La biblioteca "posee"las listas de usuarios, materiales, muebles y préstamos. Si la biblioteca dejara de existir, el contexto de estas colecciones desaparecería.

- `Library "1"*- "*"LibraryEntity`

Asociación

La clase **Loan** actúa como una clase asociativa que vincula a un `User` y un `Material` durante un periodo de tiempo.

- Cada préstamo está asociado a exactamente un usuario y un material físico.
- Un usuario puede tener múltiples préstamos (asociación 1 a *).
- Un material puede estar en múltiples historiales de préstamos (asociación 1 a *).

Diagramas UML

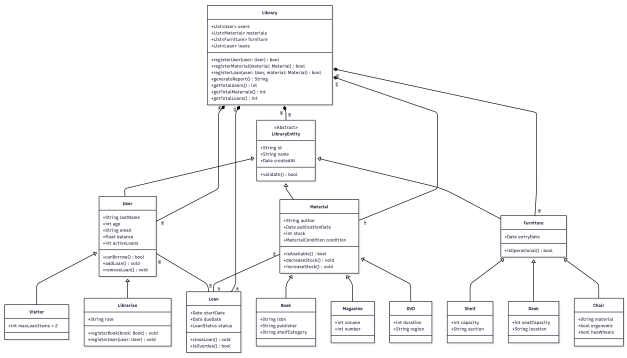

Figura 1

Diagrama de Clase UML de sistema Biblioteca

Consideración de Requisitos No Funcionales

Para garantizar la calidad y viabilidad del sistema, se han priorizado los siguientes atributos:

- **Usabilidad**: El diseño debe ser intuitivo para facilitar su operación por parte del usuario final.
- **Robustez**: El sistema debe manejar adecuadamente los errores y excepciones para evitar interrupciones críticas.